



GOBIERNO DEL
ESTADO DE
MÉXICO



EDUCACIÓN

SECRETARÍA DE EDUCACIÓN, CIENCIA, TECNOLOGÍA E INNOVACIÓN

TES VG
VILLA GUERRERO



Tecnológico de Estudios Superiores de Villa Guerrero

Ingeniería en Sistemas Computacionales

SISTEMAS OPERATIVOS

PRACTICA 1:

Explorando Procesos Con ps En Linux

ALUMNO:

Oliver Hernández Reyes

DOCENTE:

ISC Reynoso Elias Juan

Grupo: 3501

LA FINCA, VILLA GUERRERO, ESTADO DE MÉXICO.

OCTUBRE DE 2026





INTRODUCCIÓN

En los sistemas operativos basados en Linux, la gestión de procesos es un pilar fundamental para la administración eficiente del sistema, la optimización de recursos (CPU y memoria) y la supervisión de la seguridad. Un proceso representa una instancia de un programa en ejecución que requiere recursos del procesador, espacio en memoria y permisos específicos de usuario para operar.

El comando `ps` (*Process Status*) constituye la herramienta nativa de línea de comandos más utilizada para inspeccionar e diagnosticar el estado del procesador en un instante determinado. A través del análisis de este comando y sus diversas opciones de filtrado y formato (como `-ef`, `-u`, `-eo` o la canalización mediante `grep`), es posible rastrear identificadores de procesos (PID), determinar la propiedad de las ejecuciones por usuario y auditar el consumo en tiempo real de los recursos del sistema. Este reporte aborda la aplicación práctica y el análisis funcional de dichos comandos de monitoreo.





OBJETIVO

Objetivo General

Analizar y dominar el uso de la herramienta ps en entornos Linux para la inspección, filtrado y diagnóstico de procesos en ejecución a través de la interfaz de línea de comandos (CLI).

Objetivos Específicos

- Identificar las diferencias de alcance entre el monitoreo de procesos de la sesión local del usuario y la inspección global de todos los procesos del sistema.
- Aplicar filtros avanzados mediante la especificación de atributos de usuario (-u) y la canalización con grep para la localización precisa de programas específicos.
- Personalizar la salida de datos especificando columnas concretas (pid, comm, %cpu) y ordenando los resultados para detectar cuellos de botella en el procesador.





REQUISITOS

- Sistema Linux (Debian, Ubuntu, Arch, etc.)
- Terminal activa

PROCESOS PS

El comando ps (Process Status) sirve para inspeccionar los procesos que se están ejecutando en un sistema Linux. Explicación de los comandos consultados:

1. ps: Ver procesos del usuario actual

Muestra únicamente los procesos activos en la sesión de terminal actual asociados al usuario que ejecuta el comando.

- **Salida típica:** Muestra el PID (ID del proceso), la terminal (TTY), el tiempo de CPU acumulado (TIME) y el nombre del ejecutable (CMD).

```
Oliver@MACHENIKE-PC: /mnt/c/Users/Pollo$ ps
  PID TTY          TIME CMD
   202 pts/0        00:00:00 bash
   575 pts/0        00:00:00 ps
```

2. ps -ef: Ver procesos con formato extendido

Muestra **todos** los procesos en ejecución en el sistema en formato extendido.

- **-e (every):** Selecciona todos los procesos del sistema.
- **-f (full-format):** Muestra detalles adicionales como el usuario dueño (UID), el ID del proceso padre (PPID), la hora de inicio (STIME) y el comando completo con sus argumentos.



```
Oliver@MACHENIKE-PC:/mnt/c/Users/Pollo$ ps -ef
UID          PID    PPID  C  STIME TTY          TIME CMD
root          1        0  0  19:41 ?           00:00:00 /sbin/init
root          2        1  0  19:41 hvc0       00:00:00 /init
root          3        2  0  19:41 hvc0       00:00:00 init-watcher
root          7        2  0  19:41 hvc0       00:00:00 plan9 --control-socket 7 --log-lev
root         40        1  0  19:41 ?           00:00:00 /usr/lib/systemd/systemd-journald
root         78        1  0  19:41 ?           00:00:00 /usr/lib/systemd/systemd-udevd
root        156        1  0  19:41 ?           00:00:00 /usr/sbin/cron -f
message+    159        1  0  19:41 ?           00:00:00 /usr/bin/dbus-daemon --system --add
root        177        1  0  19:41 ?           00:00:00 /usr/lib/systemd/systemd-logind
root        195        1  0  19:41 tty1       00:00:00 /sbin/agetty -o -- \u --noreset --n
root        200        2  0  19:41 ?           00:00:00 /init
root        201        200  0  19:41 ?           00:00:00 /init
Oliver       202        201  0  19:41 pts/0      00:00:00 /bin/bash
root        521        2  0  20:31 ?           00:00:00 login -- Oliver
Oliver       533        1  0  20:31 ?           00:00:00 /usr/lib/systemd/systemd --user
Oliver       535        533  0  20:31 ?           00:00:00 (sd-pam)
Oliver       542        521  0  20:31 pts/1      00:00:00 -bash
Oliver       576        202  0  20:35 pts/0      00:00:00 ps -ef
```

3. ps -ef | grep Firefox: Filtrar procesos

Filtra la lista completa de procesos para buscar únicamente los que coincidan con la palabra "firefox".

- **| (pipe):** Redirige la salida de ps -ef hacia el buscador de texto.
- **grep firefox:** Filtra las líneas que contienen el término especificado.

```
Oliver@MACHENIKE-PC:/mnt/c/Users/Pollo$ ps -ef| grep firefox
Oliver       578        202  0  20:36 pts/0      00:00:00 grep firefox
```

4. ps -u nombre_usuario: Ver procesos por ID de usuario

Muestra exclusivamente los procesos que pertenecen al usuario especificado.

- **-u (user):** Filtra la lista por el nombre de usuario (o ID numérico de usuario).

```
Oliver@MACHENIKE-PC:/mnt/c/Users/Pollo$ ps -u Oliver
  PID TTY          TIME CMD
  202 pts/0      00:00:00 bash
  533 ?           00:00:00 systemd
  535 ?           00:00:00 (sd-pam)
  542 pts/1      00:00:00 bash
  586 pts/0      00:00:00 ps
```

5. ps -eo pid,comm: Ver solo PID y comando

Muestra únicamente las columnas especificadas, reduciendo el desorden en la pantalla.

- **-e:** Incluye todos los procesos.
- **-o (output format):** Define las columnas a desplegar. En este caso:





- pid: ID del proceso.
- comm: Nombre corto del comando/programa.

```
Oliver@MACHENIKE-PC:/mnt/c/Users/Pollo$ ps -eo pid,comm
PID COMMAND
  1 systemd
  2 init-systemd(De
  3 init-watcher
  7 init
 40 systemd-journal
 78 systemd-udev
156 cron
159 dbus-daemon
177 systemd-logind
195 agetty
200 SessionLeader
201 Relay(202)
202 bash
521 login
533 systemd
535 (sd-pam)
542 bash
591 ps
```

6. **ps -eo pid,comm,%cpu --sort=-%cpu:** Ver procesos ordenados por uso de CPU

Muestra la lista de todos los procesos ordenada de mayor a menor uso de procesador.

- **%cpu:** Agrega la columna con el porcentaje de uso de CPU.
- **--sort=-%cpu:** Ordena el resultado. El signo - indica orden descendente (de mayor a menor consumo).





```
Oliver@MACHENIKE-PC:/mnt/c/Users/Pollo$ ps -eo pid,comm,%cpu --sort=-%cpu
```

PID	COMMAND	%CPU
78	systemd-udevd	0.0
533	systemd	0.0
1	systemd	0.0
40	systemd-journal	0.0
177	systemd-logind	0.0
201	Relay(202)	0.0
159	dbus-daemon	0.0
156	cron	0.0
202	bash	0.0
2	init-systemd(De	0.0
3	init-watcher	0.0
7	init	0.0
195	agetty	0.0
200	SessionLeader	0.0
521	login	0.0
535	(sd-pam)	0.0
542	bash	0.0
582		0.0





CONCLUSIONES

El monitoreo de procesos mediante la CLI es una competencia esencial para cualquier administrador de sistemas o desarrollador en entornos UNIX/Linux. A través del desarrollo de esta práctica, se comprendió que el comando `ps` ofrece una visión detallada e instantánea de la actividad del sistema, permitiendo pasar de una vista general básica a un nivel de filtrado sumamente específico.

Se comprobó la versatilidad de combinar parámetros de selección de campos (`-eo`) con criterios de ordenamiento (`--sort=-%cpu`), lo cual resulta crítico para la resolución de problemas (*troubleshooting*), como la identificación inmediata de procesos colgados o con consumo excesivo de CPU. Asimismo, la integración de operadores de tubería (`|`) con utilidades de búsqueda como `grep` demostró ser el mecanismo más ágil para auditar servicios específicos en tiempo real. En conclusión, el dominio de estas variantes de `ps` proporciona un control riguroso sobre la ejecución de tareas y la administración eficiente de recursos en el sistema operativo.

